

Guia do Desenvolvedor

Setup, arquitetura e convenções

Ciclartech · Plataforma de Gestão de Ativos Assistivos

Guia do Desenvolvedor

Este guia complementa o setup rápido do `README.md` com a estrutura do projeto, convenções de código e o fluxo esperado para contribuir. Para regras de negócio, ver `docs/business-rules/` ; para arquitetura e requisitos, ver `docs/ESPECIFICACAO_TECNICA.md` .

1. Stack

Camada	Tecnologia
Linguagem	Python 3.11+
Framework web	Django 5.0
API (uso pontual)	Django REST Framework
Banco de dados	PostgreSQL 16 (local/Docker em dev; Supabase Postgres em produção)
Storage de mídia	Disco local (dev) / Supabase Storage via S3 (produção)
Hash de senha	Argon2id (<code>argon2-cffi</code>)
Servidor WSGI	Gunicorn (Docker) / adaptador <code>@vercel/python</code> (produção)
Estáticos	WhiteNoise
Testes	<code>python manage.py test</code> (Django <code>TestCase</code> , banco de teste transacional)
Lint	Ruff (<code>.ruff_cache/</code> versionado como cache, config em <code>pyproject.toml</code> / <code>ruff.toml</code> se existir)

2. Setup local

Já documentado no `README.md` (seção "Setup de desenvolvimento"). Resumo:

```
pip install -r requirements/dev.txt
cp .env.example .env
python manage.py migrate
python manage.py seed_demo
python manage.py runserver
```

`requirements/` é separado por ambiente — `base.txt` (produção, usado pelo `Dockerfile`) e `dev.txt` (acrescenta ferramentas de desenvolvimento). Adicionar uma dependência nova sempre no arquivo certo: se ela é usada em runtime de produção, vai em `base.txt` .

Usuários de teste do `seed_demo` (tenant "Prefeitura Demo", senha `demo12345`): `admin_demo` , `gestor_demo` , `func_demo` .

Editor / IDE

Não há configuração de editor versionada. Recomendado: apontar o interpretador Python do editor para o virtualenv do projeto e ativar Ruff como linter/formatter, para pegar os mesmos avisos do CI antes do commit.

3. Estrutura do projeto

Monólito modular — um único deploy Django, dividido em apps por domínio de negócio (não por camada técnica). Cada app de domínio segue o mesmo esqueleto:

```
<app>/
  models.py           # modelos Django (persistência)
  domain/             # regra de negócio pura, sem Django (só em ativos/)
  services.py         # orquestração: cria Movimentacao, dispara notificação, etc.
  selectors.py        # consultas de leitura reaproveitadas (evita N+1, centraliza
filtro)
  forms.py            # ModelForm / Form
  views.py            # view functions, decoradas com @tenant_required /
@login_required
  urls.py             # roteamento do app, incluído em ciclartech/urls.py sob
"app:<nome>"
  admin.py            # Django Admin (uso interno/depuração, não é a UI do produto)
  migrations/
  tests/
    test_models.py
    test_views.py
    test_services.py / test_state_machine.py / ...
```

App	Responsabilidade
core	Tenant , Unidade , Fornecedor , Modulo / TenantModulo (feature flags), infraestrutura de isolamento multi-tenant (TenantModel / TenantManager / core.tenancy), dashboard, relatórios, cabeçalhos de segurança (core/middleware.py)
contas	Usuario (custom, AUTH_USER_MODEL), Papel (Admin/Gestor/Funcionário), TenantMiddleware , bloqueio de login, gestão de usuários
ativos	Aggregate Ativo , máquina de estados (ativos/domain/), Movimentacao , QR Code, etiquetas, mapa operacional
beneficiarios	Beneficiario (generalizado via tipo_relacao : beneficiário/paciente/cliente), documentos anexados, LGPD (beneficiarios/lgpd.py)
notificacoes	Templates e envio (WhatsApp/Email, hoje via log estruturado)
auditoria	RegistroAuditoria append-only, middleware de captura de requisição, expurgo por retenção
owner	Área da plataforma (equipe Ciclartech): provisionamento de tenant, módulos, auditoria cross-tenant
ciclartech	Projeto Django: settings.py , urls.py , seguranca.py (validação de config de produção)

Por que domain/ só existe em ativos

A máquina de estados do Ativo (ativos/domain/state_machine.py) é a regra de negócio de maior densidade e maior reuso do produto (QR Code, ficha, painel, futura API) — por isso é isolada em Python puro, sem importar Django, e testada sem banco de dados (ativos/tests/test_state_machine.py). É "DDD cirúrgico": aplicado só onde o custo de desacoplar compensa, não um padrão a repetir em todo app.

4. Isolamento multi-tenant — a regra mais importante do código

Todo model de domínio (exceto o catálogo global core.Modulo e auditoria.RegistroAuditoria , que tem motivo próprio) herda core.models.TenantModel , que dá:

- campo tenant (FK obrigatória)
- manager objects = TenantManager() , que filtra automaticamente pelo tenant corrente da requisição (core.tenancy.get_current_tenant_id() , um ContextVar populado pelo TenantMiddleware)

Fail-closed por desenho: se não há tenant no contexto, Model.objects.all() devolve queryset **vazio**, nunca todos os registros. Um bug que esqueça de popular o contexto produz uma tela vazia (defeito visível), nunca vazamento entre tenants.

`Manager.all_tenants()` é a única forma de consulta cross-tenant — **uso exclusivo do app owner** (e do mixin de admin em `core/admin.py`), reforçado por um teste de arquitetura automatizado (`core/tests/test_architecture.py`) que falha o build se outro app chamar `all_tenants()` . Se você precisar de uma consulta cross-tenant fora desses dois lugares, é sinal de que a lógica pertence ao `owner` , não ao app que você está editando.

Fora do ciclo de request/response (management command, script, shell), o `ContextVar` não está populado — use `core.tenancy.set_current_tenant_id(tenant.pk)` / `reset_current_tenant_id(token)` num bloco `try/finally` (ver `core/management/commands/seed_demo.py` como referência), ou use `all_tenants()` com filtro explícito de tenant quando a função for chamada tanto de dentro quanto de fora de uma requisição (ver `core/unidades.py::unidades_visiveis` , `core/features.py::modulo_habilitado` — ambas documentam por que fazem isso).

5. Escopo por unidade (RBAC de segundo nível)

Além do isolamento por tenant, Gestor e Funcionário são escopados por **Unidade** (`Usuario.unidades` , M2M). Admin e Owner sempre veem todas as unidades do tenant, independente do M2M. A decisão de "o que este usuário pode ver" fica centralizada em `core/unidades.py` :

- `unidades_visiveis(usuario)` — unidades que o usuário pode operar (já aplica a regra "Admin vê tudo")
- `unidades_do_usuario(usuario)` — o M2M cru, para telas de atribuição/edição
- `filtrar_por_unidade(queryset, usuario, campo=...)` — aplica o filtro num queryset de outro model (ex.: `beneficiario_unidade` em `Notificacao`), com `incluir_sem_unidade` para registros sem unidade atribuída (visíveis a todo mundo do tenant)

Toda nova tela de listagem ou endpoint de download que exponha dado pertencente a uma Unidade precisa aplicar este filtro. Duas vulnerabilidades já corrigidas no histórico do projeto (download de documento de beneficiário e listagem de notificações que não respeitavam unidade) vieram exatamente de esquecer este passo — ver os comentários de cabeçalho de `beneficiarios/tests/test_views.py::BaixarDocumentoEscopoPorUnidadeTest` e `notificacoes/tests/test_views.py` para o histórico completo.

6. Feature flags por módulo/segmento

`core.features` decide se um tenant tem um módulo habilitado (`modulo_habilitado(tenant, codigo)`), combinando um padrão por segmento (`_MODULOS_PADRAO_POR_SEGMENTO`) com uma exceção explícita por tenant (`TenantModulo` , editável pela tela do Owner). Ver `docs/business-rules/modulos.md` para os módulos existentes (`locacao_financeiro` , `documento_pessoa_juridica`) e como adicionar um novo.

7. Testes

```
python manage.py test                                # suíte inteira
python manage.py test core.tests.test_tenancy        # um módulo
python manage.py test ativos.tests.test_state_machine.TransicaoTest.test_x # um
teste
```

Convenções observadas no repositório:

- Um `TestCase` por cenário de negócio, não por view — nomes de método em português, descrevendo o comportamento esperado (`test_funcionario_nao_pode_exportar`).
- Docstring de módulo/classe explica **por que** o teste existe quando não é óbvio (histórico da vulnerabilidade que ele previne, decisão de negócio) — não repete o que o nome do teste já diz.
- Testes de política de isolamento multi-tenant e de unidade sempre criam dois tenants/duas unidades e verificam a negativa (que o usuário do tenant/unidade B **não** aparece na resposta), não só a positiva.
- `Model.objects.all_tenants().create(...)` é o padrão em `setUp()` para popular dado de teste sem depender do `ContextVar` de tenant corrente (que só existe dentro de uma request autenticada).

Critério de saída de qualquer mudança na fundação multi-tenant ou na máquina de estados: as suítes `core/tests/test_tenancy.py`, `core/tests/test_architecture.py`, `ativos/tests/test_state_machine.py`, `ativos/tests/test_services.py` e `ativos/tests/test_acoes_disponiveis.py` têm que passar integralmente (`docs/PLANO_EVOLUCAO_SAAS_CICLARTECH.md §12`).

8. Convenções de código

- **Português para domínio, inglês para infraestrutura.** Nomes de model, campo, view, template e mensagem de usuário são em português (é o vocabulário do produto e do usuário final). Nomes de infraestrutura genérica (middleware técnico, nomes de pacote Python) seguem convenção da própria biblioteca.
- **Comentário explica o porquê, não o quê.** O código deste projeto comenta profusamente decisões de negócio/segurança não óbvias (`# ATENÇÃO: ...`, docstrings de módulo com contexto de LGPD/arquitetura) — mantenha esse padrão em vez de comentar o que a linha faz.
- **selectors.py para leitura, services.py para escrita orquestrada.** Uma view não deve montar queryset complexo inline nem criar múltiplos objetos relacionados diretamente — extraia para `selectors` / `services` do próprio app, testável isoladamente da camada HTTP.
- **Nunca lógica de negócio no template.** Cálculo de prazo, cor operacional, permissão de ação — sempre em `domain` / `services` / `selectors`, o template só exibe o que já foi decidido em Python. Ver `ativos/domain/cores.py::cor_operacional` e `ativos/domain/acoes.py` como exemplo do padrão.

- **Validação no campo do model quando a regra deve valer em qualquer caminho de escrita** (form, Admin, futura API) — não só no `Form`. Ver `core/validadores.py::validar_upload`.

9. Adicionando um novo módulo/feature flag

1. Adicionar o código em `core/features.py` (constante) e criar o registro em `Modulo` via migration de dados (mesmo padrão de `core/migrations/0003_*`, `contas/migrations/0002_seed_papeis.py`) — nunca por tela de cadastro: criar módulo é decisão de produto.
2. Se o módulo tem padrão por segmento, adicionar em `_MODULOS_PADRAO_POR_SEGMENTO`.
3. Usar `features.modulo_habilitado(tenant, CODIGO)` no form/view/template que precisa se comportar diferente.
4. Documentar em `docs/business-rules/modulos.md`.
5. Testar as duas pontas: tenant com módulo ligado vê o campo/opção; tenant sem o módulo não vê e um POST forjado não passa pela validação (ver `beneficiarios/tests/test_views.py::DocumentoFlexivelPorModuloTest` como referência do padrão de teste).

10. Adicionando um novo tipo de movimentação/transição de estado

A tabela de transições vive inteira em `ativos/domain/state_machine.py` (`_TRANSICOES_SIMPLES` / `_TRANSICOES_COM_DESTINO`). Para adicionar uma transição nova: (1) adicionar o valor em `ativos/domain/enums.py` se for um tipo de movimentação novo, (2) adicionar a entrada na tabela de transições, (3) cobrir com teste em `ativos/tests/test_state_machine.py` — inclusive o caso negativo (a transição continua proibida a partir de outros estados). Nunca decidir transição de estado na view ou no template.

11. Git / PR

- Branch a partir da branch principal, um PR por mudança logicamente coesa.
- Mensagem de commit em português, descrevendo o "porquê" quando não é óbvio pelo diff.
- CI roda `python manage.py test`, `python manage.py check --deploy --fail-level WARNING` e lint (Ruff) — ver `.github/` para o workflow exato.
- PR que altera isolamento multi-tenant, escopo de unidade ou a máquina de estados exige rodar a suíte completa localmente antes de abrir (não só o módulo tocado) — são os três pontos do sistema com maior custo de regressão silenciosa.

12. Onde encontrar o quê

Preciso de...	Vou em...
Regra de negócio de um domínio específico	docs/business-rules/<assunto>.md
Arquitetura geral, requisitos funcionais/não-funcionais	docs/ESPECIFICACAO_TECNICA.md
Modelagem do domínio de Ativos (máquina de estados, movimentação)	docs/PLANO_DOMINIO_ATIVOS.md
Roadmap SaaS (fases, papéis, segmentos, planos)	docs/PLANO_EVOLUCAO_SAAS_CICLARTECH.md
Documentação de uma feature específica já entregue	docs/features/<feature>.md
Referência de tabelas do banco	docs/REFERENCIA_BANCO_DE_DADOS.md
Deploy/infra/variáveis de ambiente	README.md (seção "Deploy na Vercel")
Segurança e LGPD implementadas	README.md (seção "Segurança e LGPD")
Operação do dia a dia da plataforma (Owner)	docs/GUIA_OPERACOES.md
Adicionar um tenant novo	docs/ONBOARDING_TENANT.md
Erro comum / debug	docs/TROUBLESHOOTING.md
Fluxo de negócio em diagrama	docs/FLUXOS_DE_NEGOCIO.md
Manual do usuário final por papel	manuals em PDF entregues por papel (Admin/Gestor/Funcionário)